

Guía de deploy: Vercel + Supabase paso a paso

De tu prototipo local a una URL pública, y de mock data a datos reales. El camino completo, sin escribir backend.

Parte 1 · Deploy en Vercel (tu URL pública)

Opción A — Desde la terminal

```
# Primera vez: instalar la CLI y loguearse (abre el browser)
npm install -g vercel
vercel login
# Desde la raíz del proyecto:
vercel                # deploy preview (URL temporal)
vercel --prod         # deploy a producción
# Vercel detecta Next.js automáticamente. Cero config.
```

Opción B — Conectando el repo de GitHub (recomendada)

Paso	Qué hacer
1	Entra a <code>vercel.com</code> y crea tu cuenta con GitHub.
2	“Add New Project” → importa tu repo de GitHub.
3	Vercel autodetecta Next.js. No configures nada.
4	“Deploy”. En 30-60 segundos tienes URL pública.
5	Desde ahora, cada <code>git push</code> a main hace deploy automático.

Tip: La opción B es la que usamos en la masterclass: convierte tu repo en una máquina de deploys. Pusheas, y en un minuto el cambio está en la URL.

Parte 2 · Supabase (cuando el prototipo pide datos reales)

Supabase te da una base de datos Postgres, autenticación y APIs generadas automáticamente. Perfecto para que los overrides del Caso 2 se guarden de verdad.

Paso 1 — Crear el proyecto

Paso	Qué hacer
1	Entra a <code>supabase.com</code> → “New project”.
2	Elige nombre, contraseña de base de datos y región (South America para Chile).

Paso	Qué hacer
3	En Settings → API, copia la Project URL y la anon key .

Paso 2 — Conectar tu app Next.js

```
npm install @supabase/supabase-js
# Crea .env.local en la raíz con tus llaves:
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
```

Ojo: `.env.local` NUNCA se commitea (ya está en `.gitignore`). En Vercel, agrega las mismas variables en Settings → Environment Variables.

Paso 3 — Crear la tabla (ejemplo Caso 2)

En Supabase → SQL Editor, pega y corre:

```
create table providers (
  id uuid default gen_random_uuid() primary key,
  name text not null,
  tin text not null unique,
  status text not null default 'vendor-check-pending',
  override_reason text,
  override_by text,
  updated_at timestamptz default now()
);
```

Paso 4 — Dirigir a Claude Code (el prompt)

Lee `claude.md` y `design.md`. Conecta la pantalla de providers a Supabase usando `@supabase/supabase-js`:

- Reemplaza el mock data por un select a la tabla providers.
- Al confirmar un override, haz update del status, el motivo y el usuario, y refresca la tabla.
- Maneja loading y error según la definición de Done.
- No expongas llaves: usa las variables de entorno.

Checklist final antes de compartir tu URL

✓	Verificación
1	<code>npm run build</code> pasa sin errores en local.
2	Variables de entorno cargadas en Vercel (y NO commiteadas).
3	La URL pública carga y el flujo completo funciona (búsqueda → override → toast).
4	Los estados loading / empty / error se ven bien con datos reales.

✓	Verificación
5	Commit final + push: el deploy automático queda registrado.

El camino completo: **mock data** → **Supabase** → **producción**. Tu prototipo deja de ser una demo y se convierte en la base del producto real que tu dev va a endurecer.